

ENGINEERING DEVELOPMENT, VALIDATION & INCIDENT REPORT

NetSentinel

Network Intelligence Platform

REPOSITORY	network-intelligence-platform
DOCUMENT TYPE	Engineering development, validation, and incident report
DEVELOPMENT PERIOD	September 2026
CURRENT LIFECYCLE STAGE	Local integration validation / pre-cloud deployment

Project maturity

Functional local integration prototype progressing toward production-ready cloud deployment.

-By Arkaprava Roy

Contents

1	Executive Summary	3
2	System Scope	3
3	High-Level Architecture	4
4	Initial Implementation.....	5
5	Development Environment and Build Validation.....	6
6	Frontend Compilation and Build Issues	7
7	Dependency Warnings	7
8	Backend Test Database Incident.....	8
9	Containerization and Local Deployment.....	9
10	Successful Docker Deployment.....	9
11	API-Level Verification.....	10
12	Synthetic Data Integration Incident	10
13	Synthetic End-to-End Validation.....	11
14	Real Windows Endpoint Integration.....	12
15	Real Endpoint Registration Validation.....	12
16	Real Telemetry Collection Investigation.....	12
17	Current Blocking Data-Type Issue	13
18	Required Remediation.....	13
19	Validation Matrix.....	14
20	Known Technical Debt and Limitations.....	15
21	Reliability Validation Plan	15
22	Security Hardening Plan.....	16
23	Cloud Deployment Strategy	16
24	Engineering Observations	17
25	Current Project State	18
26	Immediate Acceptance Criteria	18
27	Final Status.....	19

1 Executive Summary

NetSentinel is a cloud-ready network observability and intelligence platform designed to collect endpoint network and system telemetry, persist it centrally, evaluate network health, detect anomalous conditions, generate incidents, and expose the resulting intelligence through a web dashboard.

The platform has been developed and validated incrementally across the following layers:

1. Application source and dependency setup
2. Backend API and database layer
3. Automated testing
4. Containerized local deployment
5. Synthetic telemetry generation
6. Dashboard integration
7. Real Windows endpoint registration
8. Real endpoint telemetry collection

The **synthetic end-to-end pipeline is operational and visually verified**. The real Windows monitoring agent is also able to register successfully and collect a genuine host measurement.

Current blocking issue

Telemetry persistence is blocked: cumulative network byte counters can exceed the range of PostgreSQL's signed 32-bit INTEGER type. A real measurement produced a `bytes_received` value of 3,561,180,034, exceeding the INTEGER maximum of 2,147,483,647. The remediation is to migrate the relevant counters to BIGINT and validate the migration against the existing database.

No AWS resources have been deployed yet. Cloud deployment is intentionally deferred until the real endpoint-to-database telemetry path is stable.

2 System Scope

2.1 Functional Scope

NetSentinel currently encompasses:

- Endpoint registration and device identity
- Device-token authentication
- Network and system telemetry collection
- Durable local telemetry buffering
- REST telemetry ingestion
- PostgreSQL persistence
- Device health scoring
- Rule/model-based anomaly detection
- Incident generation

- React-based telemetry visualization
- Dockerized local deployment
- Synthetic demonstration data
- ML helper components
- Initial Terraform/AWS configuration
- Automated backend/agent/ML tests

2.2 Technology Stack

Layer	Technology
Endpoint agent	Python
API	FastAPI / Uvicorn
Persistence	PostgreSQL
Endpoint queue	SQLite
Frontend	React / Vite
Web serving	nginx
ML / anomaly helpers	Python — Isolation Forest and supporting feature logic
Containers	Docker / Docker Compose
Infrastructure direction	Terraform / AWS
Testing	pytest
Development OS	Windows

3 High-Level Architecture

The platform is organized into three logical layers — endpoint, application, and presentation — connected by an authenticated telemetry path and a central PostgreSQL store.

ENDPOINT LAYER

Windows / other supported host

NetSentinel Agent

Gateway | Internet | DNS | HTTP | Interfaces

CPU/RAM | Network byte counters | System information

|

v

Local SQLite Queue

| Authenticated telemetry

v

APPLICATION LAYER

FastAPI API

Registration | Telemetry | Metrics | Health | Incidents

|

v

```
PostgreSQL
|
v
Health / Anomaly Engine
|
v
PRESENTATION LAYER
React / Vite Dashboard
Devices | Health | Latency | Telemetry | Incidents | Charts
```

4 Initial Implementation

The initial implementation established the major application boundaries rather than treating the project as a single monolithic service.

4.1 Backend

The FastAPI service exposes the following endpoints:

```
GET /api/v1/health
GET /api/v1/ready

POST /api/v1/devices/register

POST /api/v1/telemetry
POST /api/v1/telemetry/batch

GET /api/v1/devices
GET /api/v1/devices/{device_id}
GET /api/v1/devices/{device_id}/metrics
GET /api/v1/devices/{device_id}/health

GET /api/v1/anomalies

GET /api/v1/incidents
GET /api/v1/incidents/{incident_id}
```

The API incorporates device-token authentication, validation, telemetry ingestion, health evaluation, and incident handling.

4.2 Endpoint Agent

The endpoint agent was designed to collect:

- Default gateway information
- Gateway latency
- Internet endpoint latency
- Packet loss
- Jitter
- DNS resolution latency

- HTTP latency
- Active network interface information
- Aggregate bytes sent/received
- CPU utilization
- Memory utilization
- Timestamp and measurement identifier

The agent maintains a local SQLite queue so measurements can survive temporary API/network failures.

4.3 Frontend

The React dashboard provides:

- Device selection
- Health score
- Latency/health time-series visualization
- Open incident count
- Incident classification
- Confidence information
- Loading and error states

4.4 Infrastructure

The local deployment consists of PostgreSQL, the FastAPI backend, and the React/nginx frontend, managed through Docker Compose.

Terraform/AWS configuration is present as a deployment starting point but has not yet been provisioned against AWS.

5 Development Environment and Build Validation

5.1 Python Environment

A project-local virtual environment was created:

```
python -m venv .venv
\\.venv\\Scripts\\Activate.ps1
```

The environment was subsequently used for backend, agent, and ML dependencies.

5.2 Node.js / npm

The first frontend validation identified that Node.js/npm were unavailable on the development machine. After installing Node.js, the environment reported:

```
Node.js: v24.21.0
npm: 11.19.0
```

This unblocked the React/Vite build pipeline.

6 Frontend Compilation and Build Issues

The initial frontend build failed with three TypeScript issues:

9. Missing lodash declaration information referenced by Recharts
10. Missing typing for import.meta.env
11. String.prototype.replaceAll unavailable under the configured TypeScript target

Remediation

- Add @types/lodash
- Add Vite client declarations
- Add a narrow Recharts/lodash compatibility declaration
- Upgrade TypeScript target/lib from ES2020 to ES2021
- Document VITE_API_URL

Validation

The strict TypeScript build subsequently passed:

```
npx tsc -b
tsc-exit=0
```

The production frontend build also passed:

```
npm run build

830 modules transformed.
built in 2.44s
```

Remaining build-time warnings concerned Vite's deprecated CJS Node API and a JavaScript chunk exceeding the recommended size threshold. Neither prevented the production build.

7 Dependency Warnings

npm reported:

```
2 vulnerabilities
1 moderate
1 high
```

A forced npm audit fix --force was intentionally not executed during stabilization because it could introduce breaking dependency upgrades. This remains a separate dependency-maintenance task.

Python test execution also produced dependency-level deprecation warnings associated with the installed FastAPI/Starlette/httpx/AnyIO stack. These did not represent application test failures.

8 Backend Test Database Incident

8.1 Initial Failure

The initial test suite produced:

```
5 tests collected
3 passed
2 failed
```

Both failures occurred during device registration and reported:

```
sqlite3.OperationalError: no such table: devices
```

8.2 Root Cause

The API tests instantiated `TestClient(app)` without entering the application's lifespan context. Consequently:

- Application startup schema initialization did not execute.
- The tests were not isolated from the normal application database session factory.

This was a test-environment lifecycle issue rather than a PostgreSQL production failure.

8.3 Remediation

- Create an isolated temporary SQLite database per test
- Create the ORM schema through `Base.metadata.create_all()`
- Override the application engine/session configuration for tests
- Enter the FastAPI lifespan correctly
- Remove the temporary test database after execution
- Pass the managed test client fixture into API tests

FastAPI schema initialization was also migrated from the deprecated startup-event pattern to a lifespan handler.

8.4 Validation

The complete relevant suite subsequently passed:

```
5 passed
```

This established a reproducible local test environment independent of any pre-existing database file.

9 Containerization and Local Deployment

9.1 Compose Invocation Error

An initial Docker Compose invocation failed with:

```
no configuration file provided: not found
```

Root Cause

The shell was positioned one directory above the repository root.

Resolution

The correct working directory was restored:

```
cd network-intelligence-platform
```

and docker-compose.yml was verified to exist.

9.2 Docker Engine Availability

The next Compose attempt failed with:

```
failed to connect to the docker API
```

referencing the Docker Desktop Linux engine.

Root Cause

Docker Desktop's engine was not running.

Resolution

Docker Desktop was started, after which the Docker daemon became available.

10 Successful Docker Deployment

The stack was successfully launched using:

```
docker compose up --build
```

The services reported healthy startup.

PostgreSQL

```
database system is ready to accept connections
```

FastAPI

```
Application startup complete.  
Uvicorn running on http://0.0.0.0:8000
```

Frontend

The nginx frontend container completed its startup sequence successfully. docker compose ps confirmed the following local exposure:

Service	Port	State
Backend	8000	Running
Frontend	5173	Running
PostgreSQL	5432	Running / healthy

11 API-Level Verification

The FastAPI OpenAPI/Swagger interface was successfully accessed at:

```
http://localhost:8000/docs
```

The application health endpoint returned:

```
{  
  "status": "healthy"  
}
```

through GET /api/v1/health.

The root /health endpoint returned HTTP 404 because that route is not part of the implemented API contract.

12 Synthetic Data Integration Incident

12.1 Initial Symptom

The dashboard initially displayed:

```
No devices yet. Register an agent or run the seed script.
```

The synthetic seed script reported successful execution, but GET /api/v1/devices returned:

```
[]
```

12.2 Root Cause

The host-side seed script defaulted to:

```
sqlite:///./netsentinel.db
```

while the Dockerized backend used PostgreSQL:

```
postgresql+psycopg://netsentinel:...@postgres:5432/netsentinel
```

The seed process and application were therefore operating against different database instances.

12.3 Remediation

The demo workflow was updated to make the database target explicit.

scripts/seed_demo_data.py

Added a --database-url option and ensured the selected URL is applied before loading the application database module. Connection output is password-redacted.

docker-compose.yml

Added a dedicated seed service under the demo profile, targeting the same PostgreSQL service used by the backend.

README.md

Documented both Docker-native and host-side demo seeding workflows.

12.4 Validation

Synthetic telemetry was successfully inserted into the Docker PostgreSQL database:

```
80 telemetry records
```

The live API subsequently returned the seeded device and its metrics.

13 Synthetic End-to-End Validation

The corrected synthetic pipeline was verified through the live application. The dashboard displayed:

- Demo Wi-Fi device
- Health score
- Telemetry time-series
- Open incident
- Incident classification
- Model/rule confidence

An example generated incident was UPSTREAM DEGRADATION, with 67% model/rule confidence and status OPEN. The evidence presented by the application indicated normal gateway behavior alongside increased Internet latency.

This validated the following path:

```
Synthetic measurement
|
v
PostgreSQL -> FastAPI -> Health/anomaly processing -> Incident generation -> React dashboard
```

Result

The synthetic pipeline is considered functionally validated in the local environment.

14 Real Windows Endpoint Integration

The next validation stage replaced synthetic telemetry with a real endpoint. A registration workflow was added to the agent:

```
python -m netsentinel_agent.main --register --name $env:COMPUTERNAME
```

The registration response provides `device_id` and `device_token`. Telemetry uploads authenticate using the X-Device-Token header.

The generated token is a credential and is intentionally excluded from source control and public documentation.

15 Real Endpoint Registration Validation

The Windows host successfully registered as **ARKADESKTOP** with platform windows. The device appeared in the live dashboard alongside the synthetic device. This confirmed that the following path is functioning:

```
Windows Agent -> Device Registration API -> PostgreSQL -> React Device Selector
```

16 Real Telemetry Collection Investigation

A single-cycle collection was executed:

```
python -m netsentinel_agent.main --once
```

The initial API checks returned:

```
GET /api/v1/devices/{device_id}/metrics  
→ []  
  
GET /api/v1/devices/{device_id}/health  
→ {"detail": "no telemetry"}
```

Further investigation of the agent's durable queue established that the situation was not a collection failure. The agent had:

12. Collected a genuine Windows host measurement
13. Serialized the measurement
14. Attempted the upload
15. Retried the upload
16. Marked the upload as failed

This narrowed the failure to the persistence/upload path.

17 Current Blocking Data-Type Issue

The real measurement contained:

```
bytes_received = 3,561,180,034
```

The current PostgreSQL telemetry representation uses a signed 32-bit INTEGER. The maximum value of that type is:

2,147,483,647

Therefore 3,561,180,034 exceeds 2,147,483,647, and the database cannot persist the measurement.

Failure path

Stage	Result
Telemetry collection	PASS
Payload construction	PASS
Upload attempt	PASS
PostgreSQL persistence	FAIL
Metrics API	[] (empty)
Health calculation	Unavailable
Primary blocker This is currently the primary technical blocker to completing the real endpoint-to-dashboard pipeline.	

18 Required Remediation

The cumulative byte counters should use PostgreSQL **BIGINT** rather than **INTEGER**. Affected fields:

```
bytes_sent
bytes_received
```

The required implementation should include:

- Updating the SQLAlchemy model to use BigInteger
- Applying an idempotent PostgreSQL schema migration
- Preserving all existing records
- Maintaining compatibility with the agent's local SQLite queue
- Verifying existing Docker PostgreSQL data remains intact
- Improving upload error reporting so failures are observable rather than silently swallowed

The migration must not:

- Truncate telemetry
- Reset PostgreSQL
- Clamp counter values
- Generate synthetic replacements
- Modify frontend behavior to hide the failure

19 Validation Matrix

Capability	Status	Evidence
Python environment	Verified	Project virtual environment operational
Node.js/npm	Verified	Node v24.21.0, npm 11.19.0
TypeScript compilation	Verified	tsc -b passed
Frontend production build	Verified	Vite build passed
Backend/agent/ML tests	Verified	5 passed
Docker Compose	Verified	All services running
PostgreSQL	Verified	Healthy and accepting connections
FastAPI	Verified	/api/v1/health returned healthy
OpenAPI documentation	Verified	/docs accessible
Synthetic seed	Verified	80 records inserted
Synthetic dashboard	Verified	Device, chart, health and incident displayed
Device registration	Verified	ARCADESKTOP registered
Real telemetry collection	Verified	Genuine host measurement created
Real telemetry upload	Partially verified	Upload attempted; DB persistence failed
Real telemetry persistence	Blocked	INTEGER overflow
Real telemetry dashboard	Pending	Requires schema correction
Reliability testing	Pending	Not yet completed
Security hardening	Pending	Not yet completed
AWS deployment	Pending	Not yet provisioned

20 Known Technical Debt and Limitations

Frontend

- Vite CJS Node API deprecation warning
- Production bundle contains a chunk above the recommended size threshold
- Dependency vulnerability review remains outstanding

Agent

- Upload retry currently lacks exponential backoff
- YAML configuration is not currently loaded by the implementation
- Network speed testing is not currently implemented

- Upload error observability should be improved

Infrastructure

- PostgreSQL is exposed on host port 5432 in the development Compose configuration
- Production deployment should isolate the database from public access
- Cloud deployment has not yet been executed

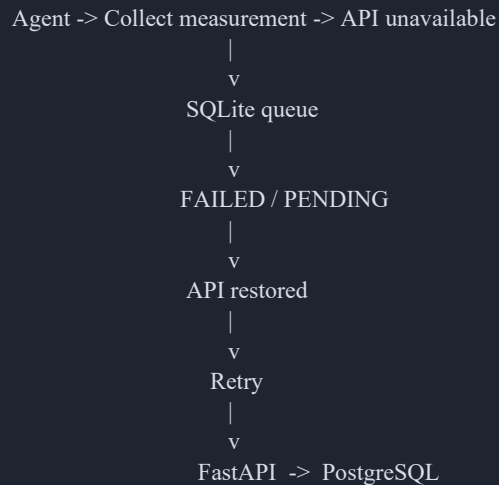
Testing

The current automated suite validates core backend/agent/ML behavior, but broader integration and failure-recovery testing remains necessary.

21 Reliability Validation Plan

Before cloud deployment, the agent's durability mechanism should be tested deliberately.

Target failure/recovery path



The following scenarios should be validated:

- API unavailable
- Internet unavailable
- DNS unavailable
- Gateway unreachable
- Invalid API response
- Authentication failure
- Repeated upload failures
- Queue recovery after agent restart
- Retry-limit behavior
- Duplicate measurement handling

The objective is to demonstrate that temporary connectivity failures do not result in silent telemetry loss.

22 Security Hardening Plan

Before exposing the platform beyond the local development environment:

- Rotate development credentials
- Move secrets to environment/secret management
- Review device-token lifecycle
- Restrict CORS
- Add API rate limiting where appropriate
- Review request validation
- Verify SQLAlchemy query safety
- Remove unnecessary database exposure
- Review container privileges
- Configure HTTPS/TLS
- Separate development and production configuration
- Configure centralized logging
- Review cloud security groups and network policy

23 Cloud Deployment Strategy

AWS deployment is intentionally the final infrastructure phase rather than the current development target. The intended progression is:

Local validation -> Real endpoint validation -> Reliability testing -> Security hardening -> Production containerization -> AWS deployment

A future cloud architecture can preserve the same logical boundaries:

```
Endpoint Agent
  | HTTPS
  v
AWS Application/API Layer
  |
  v
Managed PostgreSQL
  |
  v
Health / Anomaly / Incident Processing
  |
  v
Web Dashboard
```

Terraform configuration can subsequently be used to make the infrastructure reproducible.

24 Engineering Observations

The validation process demonstrated several important engineering principles.

Build success is not system success

The frontend compiled successfully while integration issues remained elsewhere in the stack.

Application lifecycle matters in tests

The missing SQLite tables were caused by incorrect test lifecycle handling rather than a production schema defect.

Host and container configuration must be explicit

The synthetic seeding failure occurred because the host process and Docker backend silently selected different database targets.

End-to-end validation exposes production-relevant constraints

The real endpoint test uncovered a data-type limitation that synthetic telemetry did not expose.

AI-assisted implementation still requires independent verification

Coding assistance accelerated implementation, but major milestones were validated independently using:

- Build commands
- Automated tests
- Docker health
- Live API requests
- Database inspection
- Browser verification
- Real endpoint execution

The project is therefore being treated as an engineering system rather than an unverified generated codebase.

25 Current Project State

Operational

- Local Docker environment
- PostgreSQL
- FastAPI backend
- React dashboard
- Synthetic telemetry pipeline
- Device registration
- Health/anomaly/incident visualization
- Automated test suite
- Real Windows endpoint collection

In progress

- Real endpoint telemetry persistence
- Database schema correction

- Upload error observability
- Reliability validation

Not yet started

- Production cloud deployment
- Production security configuration
- Infrastructure provisioning
- Full CI/CD validation
- Production monitoring and alerting

26 Immediate Acceptance Criteria

The next milestone is deliberately narrow:

Milestone

Persist one genuine ARKADESKTOP telemetry measurement in PostgreSQL and render it in the dashboard.

Acceptance criteria

- ☐ PostgreSQL byte-counter columns migrated to BIGINT
- ☐ Existing database data preserved
- ☐ Agent performs --once collection
- ☐ Agent successfully uploads telemetry
- ☐ API returns ARKADESKTOP telemetry
- ☐ Health endpoint calculates a score
- ☐ PostgreSQL contains the measurement
- ☐ Dashboard renders ARKADESKTOP telemetry
- ☐ Existing automated tests still pass

Only after these criteria are satisfied should the project proceed to reliability testing and cloud deployment.

27 Final Status

NetSentinel has progressed beyond a static prototype. The project currently has:

- a validated local architecture,
- a working synthetic end-to-end pipeline,
- a functioning dashboard,
- a Dockerized backend/database stack,
- a working device-registration flow, and
- a real Windows endpoint capable of collecting genuine network/system measurements.

The remaining immediate blocker is a schema mismatch between real-world cumulative network byte counters and the current PostgreSQL integer representation.

The next engineering task is therefore a targeted schema migration from INTEGER to BIGINT, followed by live validation of the real endpoint telemetry path.

Status at report time

Item	Status
Local platform	Verified
Synthetic E2E pipeline	Verified
Real endpoint registration	Verified
Real telemetry collection	Verified
Real telemetry persistence	Blocked — BIGINT migration required
Real telemetry dashboard	Pending
Reliability testing	Pending
Security hardening	Pending
AWS deployment	Pending

Project maturity

Functional local integration prototype progressing toward production-ready cloud deployment.